JavaScript is disabled on your browser.

compact1, compact2, compact3

java.time

Class ZoneOffset

java.lang.Object
    java.time.ZoneId
        java.time.ZoneOffset

All Implemented Interfaces:

`Serializable`, `Comparable<ZoneOffset>`, `TemporalAccessor`, `TemporalAdjuster`

```
public final class ZoneOffset
extends ZoneId
implements TemporalAccessor, TemporalAdjuster, Comparable<ZoneOffset>, Serializable
```

A time-zone offset from Greenwich/UTC, such as `+02:00`.

A time-zone offset is the amount of time that a time-zone differs from Greenwich/UTC. This is usually a fixed number of hours and minutes.

Different parts of the world have different time-zone offsets. The rules for how offsets vary by place and time of year are captured in the `ZoneId` class.

For example, Paris is one hour ahead of Greenwich/UTC in winter and two hours ahead in summer. The `ZoneId` instance for Paris will reference two `ZoneOffset` instances - a `+01:00` instance for winter, and a `+02:00` instance for summer.

In 2008, time-zone offsets around the world extended from -12:00 to +14:00. To prevent any problems with that range being extended, yet still provide validation, the range of offsets is restricted to -18:00 to 18:00 inclusive.

This class is designed for use with the ISO calendar system. The fields of hours, minutes and seconds make assumptions that are valid for the standard ISO definitions of those fields. This class may be used with other calendar systems providing the definition of the time fields matches those of the ISO calendar system.

Instances of ZoneOffset must be compared using equals(java.lang.Object). Implementations may choose to cache certain common offsets, however applications must not rely on such caching.

This is a value-based class; use of identity-sensitive operations (including reference equality (==), identity hash code, or synchronization) on instances of ZoneOffset may have unpredictable results and should be avoided. The equals method should be used for comparisons.

Implementation Requirements:

This class is immutable and thread-safe.

Since:

1.8

See Also:

Serialized Form

Field Summary

Fields

Modifier and Type	Field	Description
static ZoneOffset	MAX	Constant for the maximum supported offset.
static ZoneOffset	MIN	Constant for the maximum supported offset.
static ZoneOffset	UTC	The time-zone offset for UTC, with an ID of 'Z'.

Fields inherited from class java.time.ZoneId

SHORT_IDS

Method Summary

Instance Methods | **Concrete Methods**

Modifier and Type	Method	Description
Temporal	adjustInto(Temporal temporal)	Adjusts the specified temporal

		object to have the same offset as this object.
int	**compareTo**(**ZoneOffset** other)	Compares this offset to another offset in descending order.
boolean	**equals**(**Object** obj)	Checks if this offset is equal to another offset.
static **ZoneOffset**	**from**(**TemporalAccessor** temporal)	Obtains an instance of `ZoneOffset` from a temporal object.
int	**get**(**TemporalField** field)	Gets the value of the specified field from this offset as an `int`.
String	**getId**()	Gets the normalized zone offset ID.
long	**getLong**(**TemporalField** field)	Gets the value of the specified field from this offset as a `long`.
ZoneRules	**getRules**()	Gets the associated time-zone rules.
int	**getTotalSeconds**()	Gets the total zone offset in seconds.
int	**hashCode**()	A hash code for this offset.

boolean	**isSupported**(**TemporalField** field)	Checks if the specified field is supported.
static **ZoneOffset**	**of**(**String** offsetId)	Obtains an instance of ZoneOffset using the ID.
static **ZoneOffset**	**ofHours**(int hours)	Obtains an instance of ZoneOffset using an offset in hours.
static **ZoneOffset**	**ofHoursMinutes**(int hours, int minutes)	Obtains an instance of ZoneOffset using an offset in hours and minutes.
static **ZoneOffset**	**ofHoursMinutesSeconds**(int hours, int minutes, int seconds)	Obtains an instance of ZoneOffset using an offset in hours, minutes and seconds.
static **ZoneOffset**	**ofTotalSeconds**(int totalSeconds)	Obtains an instance of ZoneOffset specifying the total offset in seconds
<R> R	**query**(**TemporalQuery**<R> query)	Queries this offset using the specified query.
ValueRange	**range**(**TemporalField** field)	Gets the range of valid values for the specified field.
String	**toString**()	Outputs this offset as a

		`String`, using the normalized ID.

Methods inherited from class java.time.ZoneId

`getAvailableZoneIds`, `getDisplayName`, `normalized`, `of`, `ofOffset`, `systemDefault`

Methods inherited from class java.lang.Object

`clone`, `finalize`, `getClass`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Field Detail

UTC

```
public static final ZoneOffset UTC
```

The time-zone offset for UTC, with an ID of 'Z'.

MIN

```
public static final ZoneOffset MIN
```

Constant for the maximum supported offset.

MAX

```
public static final ZoneOffset MAX
```

Constant for the maximum supported offset.

Method Detail

of

```
public static ZoneOffset of(String offsetId)
```

Obtains an instance of `ZoneOffset` using the ID.

This method parses the string ID of a `ZoneOffset` to return an instance. The parsing accepts all the formats generated by `getId()`, plus some additional formats:

- `Z` - for UTC
- `+h`
- `+hh`
- `+hh:mm`
- `-hh:mm`
- `+hhmm`
- `-hhmm`
- `+hh:mm:ss`
- `-hh:mm:ss`
- `+hhmmss`
- `-hhmmss`

Note that ± means either the plus or minus symbol.

The ID of the returned offset will be normalized to one of the formats described by `getId()`.

The maximum supported range is from +18:00 to -18:00 inclusive.

Parameters:

`offsetId - the offset ID, not null`

Returns:

`the zone-offset, not null`

Throws:

`DateTimeException - if the offset ID is invalid`

ofHours

```
public static ZoneOffset ofHours(int hours)
```

Obtains an instance of `ZoneOffset` using an offset in hours.

Parameters:

`hours - the time-zone offset in hours, from -18 to +18`

Returns:

`the zone-offset, not null`

Throws:

`DateTimeException - if the offset is not in the required range`

ofHoursMinutes

```
public static ZoneOffset ofHoursMinutes(int hours,
                                        int minutes)
```

Obtains an instance of `ZoneOffset` using an offset in hours and minutes.

The sign of the hours and minutes components must match. Thus, if the hours is negative, the minutes must be negative or zero. If the hours is zero, the minutes may be positive, negative or zero.

Parameters:

`hours - the time-zone offset in hours, from -18 to +18`

`minutes - the time-zone offset in minutes, from 0 to ±59, sign matches hours`

Returns:

`the zone-offset, not null`

Throws:

`DateTimeException - if the offset is not in the required range`

ofHoursMinutesSeconds

```
public static ZoneOffset ofHoursMinutesSeconds(int hours,
                                               int minutes,
                                               int seconds)
```

Obtains an instance of `ZoneOffset` using an offset in hours, minutes and seconds.

The sign of the hours, minutes and seconds components must match. Thus, if the hours is negative, the minutes and seconds must be negative or zero.

Parameters:

`hours - the time-zone offset in hours, from -18 to +18`

`minutes - the time-zone offset in minutes, from 0 to ±59, sign matches hours and seconds`

`seconds - the time-zone offset in seconds, from 0 to ±59, sign matches hours and minutes`

Returns:

`the zone-offset, not null`

Throws:

`DateTimeException - if the offset is not in the required range`

from

`public static ZoneOffset from(TemporalAccessor temporal)`

Obtains an instance of `ZoneOffset` from a temporal object.

This obtains an offset based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `ZoneOffset`.

A `TemporalAccessor` represents some form of date and time information. This factory converts the arbitrary temporal object to an instance of `ZoneOffset`.

The conversion uses the `TemporalQueries.offset()` query, which relies on extracting the `OFFSET_SECONDS` field.

This method matches the signature of the functional interface `TemporalQuery` allowing it to be used as a query via method reference, `ZoneOffset::from`.

Parameters:

`temporal - the temporal object to convert, not null`

Returns:

`the zone-offset, not null`

Throws:

`DateTimeException - if unable to convert to an ZoneOffset`

ofTotalSeconds

`public static ZoneOffset ofTotalSeconds(int totalSeconds)`

Obtains an instance of `ZoneOffset` specifying the total offset in seconds

The offset must be in the range `-18:00` to `+18:00`, which corresponds to -64800 to +64800.

Parameters:

`totalSeconds - the total time-zone offset in seconds, from -64800 to +64800`

Returns:

`the ZoneOffset, not null`

Throws:

`DateTimeException - if the offset is not in the required range`

getTotalSeconds

```
public int getTotalSeconds()
```

Gets the total zone offset in seconds.

This is the primary way to access the offset amount. It returns the total of the hours, minutes and seconds fields as a single offset that can be added to a time.

Returns:

the total zone offset amount in seconds

getId

```
public String getId()
```

Gets the normalized zone offset ID.

The ID is minor variation to the standard ISO-8601 formatted string for the offset. There are three formats:

- `Z` - for UTC (ISO-8601)
- `+hh:mm` or `-hh:mm` - if the seconds are zero (ISO-8601)
- `+hh:mm:ss` or `-hh:mm:ss` - if the seconds are non-zero (not ISO-8601)

Specified by:

`getId` in class `ZoneId`

Returns:

the zone offset ID, not null

getRules

```
public ZoneRules getRules()
```

Gets the associated time-zone rules.

The rules will always return this offset when queried. The implementation class is immutable, thread-safe and serializable.

Specified by:

`getRules` in class `ZoneId`

Returns:

the rules, not null

isSupported

```
public boolean isSupported(TemporalField field)
```

Checks if the specified field is supported.

This checks if this offset can be queried for the specified field. If false, then calling the `range` and `get` methods will throw an exception.

If the field is a `ChronoField` then the query is implemented here. The `OFFSET_SECONDS` field returns true. All other `ChronoField` instances will return false.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.isSupportedBy(TemporalAccessor)` passing `this` as the argument. Whether the field is supported is determined by the field.

Specified by:

`isSupported` in interface `TemporalAccessor`

Parameters:

`field - the field to check, null returns false`

Returns:

`true if the field is supported on this offset, false if not`

range

```
public ValueRange range(TemporalField field)
```

Gets the range of valid values for the specified field.

The range object expresses the minimum and maximum valid values for a field. This offset is used to enhance the accuracy of the returned range. If it is not possible to return the range, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `supported fields` will return appropriate range instances. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.rangeRefinedBy(TemporalAccessor)` passing `this` as the argument. Whether the range can be obtained is determined by the field.

Specified by:

`range` in interface `TemporalAccessor`

Parameters:

field - the field to query the range for, not null

Returns:

the range of valid values for the field, not null

Throws:

DateTimeException - if the range for the field cannot be obtained

UnsupportedTemporalTypeException - if the field is not supported

get

public int get(TemporalField field)

Gets the value of the specified field from this offset as an int.

This queries this offset for the value of the specified field. The returned value will always be within the valid range of values for the field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a ChronoField then the query is implemented here. The OFFSET_SECONDS field returns the value of the offset. All other ChronoField instances will throw an UnsupportedTemporalTypeException.

If the field is not a ChronoField, then the result of this method is obtained by invoking TemporalField.getFrom(TemporalAccessor) passing this as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

get in interface TemporalAccessor

Parameters:

field - the field to get, not null

Returns:

the value for the field

Throws:

DateTimeException - if a value for the field cannot be obtained or the value is outside the range of valid values for the field

UnsupportedTemporalTypeException - if the field is not supported or the range of values exceeds an int

ArithmeticException - if numeric overflow occurs

getLong

```
public long getLong(TemporalField field)
```

Gets the value of the specified field from this offset as a `long`.

This queries this offset for the value of the specified field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `OFFSET_SECONDS` field returns the value of the offset. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.getFrom(TemporalAccessor)` passing `this` as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

`getLong` in interface `TemporalAccessor`

Parameters:

`field` - the field to get, not null

Returns:

the value for the field

Throws:

`DateTimeException` - if a value for the field cannot be obtained

`UnsupportedTemporalTypeException` - if the field is not supported

`ArithmeticException` - if numeric overflow occurs

query

```
public <R> R query(TemporalQuery<R> query)
```

Queries this offset using the specified query.

This queries this offset using the specified query strategy object. The `TemporalQuery` object defines the logic to be used to obtain the result. Read the documentation of the query to understand what the result of this method will be.

The result of this method is obtained by invoking the `TemporalQuery.queryFrom(TemporalAccessor)` method on the specified query passing `this` as the argument.

Specified by:

query in interface TemporalAccessor

Type Parameters:

R - the type of the result

Parameters:

query - the query to invoke, not null

Returns:

the query result, null may be returned (defined by the query)

Throws:

DateTimeException - if unable to query (defined by the query)

ArithmeticException - if numeric overflow occurs (defined by the query)

adjustInto

```
public Temporal adjustInto(Temporal temporal)
```

Adjusts the specified temporal object to have the same offset as this object.

This returns a temporal object of the same observable type as the input with the offset changed to be the same as this.

The adjustment is equivalent to using Temporal.with(TemporalField, long) passing ChronoField.OFFSET_SECONDS as the field.

In most cases, it is clearer to reverse the calling pattern by using Temporal.with(TemporalAdjuster):

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisOffset.adjustInto(temporal);
temporal = temporal.with(thisOffset);
```

This instance is immutable and unaffected by this method call.

Specified by:

adjustInto in interface TemporalAdjuster

Parameters:

temporal - the target object to be adjusted, not null

Returns:

the adjusted object, not null

Throws:

DateTimeException - if unable to make the adjustment

ArithmeticException - if numeric overflow occurs

compareTo

```
public int compareTo(ZoneOffset other)
```

Compares this offset to another offset in descending order.

The offsets are compared in the order that they occur for the same time of day around the world. Thus, an offset of `+10:00` comes before an offset of `+09:00` and so on down to `-18:00`.

The comparison is "consistent with equals", as defined by `Comparable`.

Specified by:

`compareTo` in interface `Comparable<ZoneOffset>`

Parameters:

`other` - the other date to compare to, not null

Returns:

the comparator value, negative if less, positive if greater

Throws:

`NullPointerException` - if other is null

equals

```
public boolean equals(Object obj)
```

Checks if this offset is equal to another offset.

The comparison is based on the amount of the offset in seconds. This is equivalent to a comparison by ID.

Overrides:

`equals` in class `ZoneId`

Parameters:

`obj` - the object to check, null returns false

Returns:

true if this is equal to the other offset

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

A hash code for this offset.

Overrides:

`hashCode` in class `ZoneId`

Returns:

a suitable hash code

See Also:

`Object.equals(java.lang.Object)`, `System.identityHashCode(java.lang.Object)`

toString

```
public String toString()
```

Outputs this offset as a `String`, using the normalized ID.

Overrides:

`toString` in class `ZoneId`

Returns:

a string representation of this offset, not null

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.